Лекция №2.
MPI. Подробнее.



Григорьев Василий Васильевич

2023



Функции коллективного взаимодействия процессов

Одной из наиболее привлекательных сторон MPI является наличие широкого набора коллективных операций, которые берут на себя выполнение наиболее часто встречающихся при программировании действий. Например, часто возникает потребность разослать некоторую переменную или массив из одного процессора всем остальным. Каждый программист может написать такую процедуру с использованием операций Send/Recv, однако гораздо удобнее воспользоваться коллективной операцией MPI_Bcast. Причем гарантировано, что эта операция будет выполняться гораздо эффективнее, поскольку MPI - функция реализована с использованием внутренних возможностей коммуникационной среды.

```
MPI_Comm_size(MPI_COMM_WORLD,&size);
for (i=1; i<size; i++)
  MPI_Send(&buf,buflen,MPI_INT,i,0,MPI_COMM_WORLD);
```

Пример MPI_Bcast

```c
#include <stdio.h>
#include <mpi.h>
int main() {
  int rank, size;
  int message[5];
  MPI_Init(NULL, NULL);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  if (rank == 0) {
    // Процесс-корень заполняет массив сообщения
    for (int i = 0; i < 5; i++) {
      message[i] = i;
    }
    printf("Процесс-корень: %d, отправлено сообщение:", rank);
    for (int i = 0; i < 5; i++) {
      printf("%d ", message[i]);
    }
    printf("\n");
  }
  // Распространение массива сообщения от процесса-корня всем остальным процессам
  MPI_Bcast(message, 5, MPI_INT, 0, MPI_COMM_WORLD);
  printf("Процесс: %d, получено сообщение: ", rank);
  for (int i = 0; i < 5; i++) {
    printf("%d ", message[i]);
  }
  printf("\n");
  MPI_Finalize();
  return 0;
}
```

Он отправляет еще и самому себе

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислительные системы и параллельная обработка данных/Программы/MPI$ mpicc bcast.c -o bcast
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислительные системы и параллельная обработка данных/Программы/MPI$ mpirun -np 8 bcast
Процесс: 5, получено сообщение: 0 1 2 3 4
Процесс-корень: 0, отправлено сообщение:0 1 2 3 4
Процесс: 0, получено сообщение: 0 1 2 3 4
Процесс: 1, получено сообщение: 0 1 2 3 4
Процесс: 6, получено сообщение: 0 1 2 3 4
Процесс: 2, получено сообщение: 0 1 2 3 4
Процесс: 4, получено сообщение: 0 1 2 3 4
Процесс: 7, получено сообщение: 0 1 2 3 4
Процесс: 3, получено сообщение: 0 1 2 3 4
```

Широковещательная рассылка данных

Широковещательная рассылка данных выполняется с помощью функции MPI_Bcast. Процесс с номером root рассылает сообщение из своего буфера передачи всем процессам области связи коммуникатора comm.

```
int MPI_Bcast(void* buffer, int count, MPI_Datatype datatype, int root,
MPI_Comm comm )
```

buffer - адрес начала расположения в памяти рассылаемых данных;

count - число посылаемых элементов;

datatype - тип посылаемых элементов;

root - номер процесса-отправителя;

comm - коммуникатор.

После завершения подпрограммы каждый процесс в области связи коммуникатора comm, включая и самого отправителя, получит копию сообщения от процесса-отправителя root.

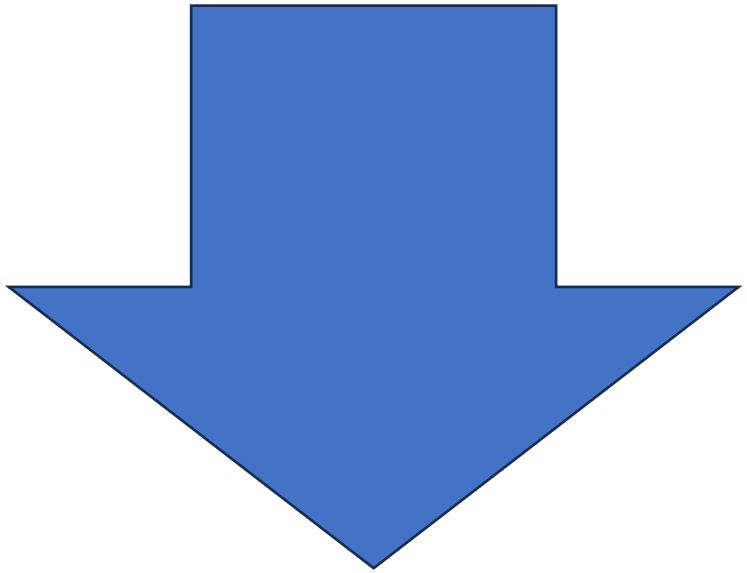
Набор коллективных операций

Функция синхронизации процессов MPI_Barrier блокирует работу вызвавшего ее процесса до тех пор, пока все другие процессы группы также не вызовут эту функцию. Завершение работы этой функции возможно только всеми процессами одновременно (все процессы "преодолевают барьер" одновременно)

int MPI_Barrier(MPI_Comm comm)

Пример

Пусть нам требуется вычислить скалярное произведение двух векторов, причем вектора вводятся на нулевом процессе (например, считываются из файла).



Пример

```
#include "mpi.h"
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
double x[100], y[100];
double res, p_res = 0.0;
 MPI_Status status;
 int n, myid, numprocs, i, N;
 N = 100;
 /* инициализация MPI */
 /* выяснение числа процессов и рангов */
 MPI_Init(&argc,&argv);
 MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
 MPI_Comm_rank(MPI_COMM_WORLD,&myid);
/* ввод векторов */
if (0 == myid) read_vectors(x,y);
/*рассылка векторов на все процессы*/
MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(y, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
/*вычисление частичной суммы на каждом из процессов*/
for (i = myid * N/numprocs; i < (myid + 1)*N/
numprocs; i++)
 p_res = p_res + x[i] * y[i];
 /* сборка на 0-м процессе результата*/
if (0 == myid){
for (i=1; i< numprocs; i++)
 MPI_Recv(&res, 1, MPI_DOUBLE, i,
MPI_ANY_TAG,
 MPI_COMM_WORLD, &status);
 p_res = p_res+res;

}else /*все процессы отсылают частичные суммы на 0-й*/
MPI_Send(&p_res,1,MPI_DOUBLE,0,0,MPI_COMM_WORLD
);
 /*на 0-м печатаем результат*/
if (0 == myid) printf("\nInner product = %10.2f",p_res);
MPI_Finalize();
}
```

Набор коллективных операций

Все коммуникационные подпрограммы, за исключением MPI_Bcast, представлены в двух вариантах:

- простой вариант, когда все части передаваемого сообщения имеют одинаковую длину и занимают смежные области в адресном пространстве процессов;
- "векторный" вариант, который предоставляет более широкие возможности по организации коллективных коммуникаций, снимая ограничения, присущие простому варианту, как в части длин блоков, так и в части размещения данных в адресном пространстве процессов. Векторные варианты отличаются дополнительным символом "v" в конце имени функции.

Передача данных от всех процессов одному. Операции редукции

Приведенный выше код решает эту проблему неэффективно - с помощью уже знакомого цикла приема сообщений от всех процессов. MPI предоставляет возможность решить эту задачу по-другому - используя обратную по отношению к широковещательной рассылке операцию - операцию сбора данных или редукцию. Операция редукции позволяет, собрав на одном из узлов данные, посланные остальными узлами, выполнить над ними какую-либо из групповых операций - типа сложения, поиска максимума, минимума, среднего значения и т.д

Что?

int MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)

– sendbuf - данные, которые посылаются на каждом из процессов;
– recvbuf - адрес буфера, в который будет помещен результат;
– count - количество элементов в буфере sendbuf;
– datatype - тип элементов в буфере sendbuf;
– op - коллективная операция, которая должна быть выполнена над данными;
– root - ранг процесса, на котором должен быть собран результат;
– comm - коммуникатор, в рамках которого производится коллективная операция.

Что?

В качестве коллективных операций можно использовать предопределенные в MPI операции, такие как MPI_SUM, MPI_MIN, MPI_MAX и другие (всего их 12). Кроме того, имеется возможность определить свою операцию (определение пользовательской операции выполняется с помощью функции MPI_Op_create - рассмотрение этого вопроса выходит за рамки нашего курса)

Пример

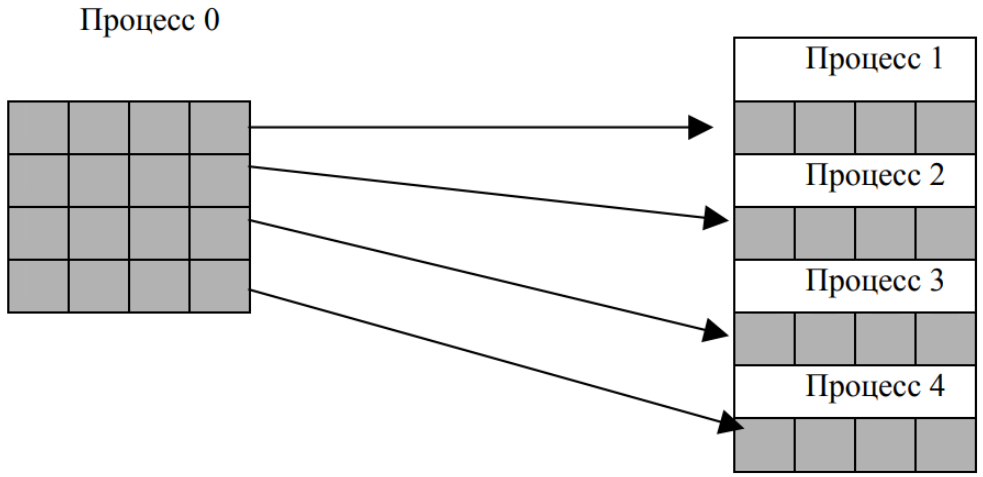
```
#include "mpi.h"
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {

double x[100], y[100];
double res, p_res = 0.0;
MPI_Status status;
int n, myid, numprocs, i, N;
N = 100;
 /* инициализация MPI */
/* выяснение числа процессов и рангов */
 MPI_Init(&argc,&argv);
 MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
 MPI_Comm_rank(MPI_COMM_WORLD,&myid);
```

```
/* ввод векторов */
if (0 == myid) read_vectors(x,y);
/*рассылка векторов на все процессы*/
MPI_Bcast(x, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Bcast(y, N, MPI_DOUBLE, 0, MPI_COMM_WORLD);
/*вычисление частичной суммы на каждом из процессов*/
for (i = myid *N/numprocs; i < (myid + 1)*N/numprocs;
i++)
 p_res = p_res + x[i] * y[i];
/* сборка на 0-м процессе результата*/
MPI_Reduce(&p_res, &res, N/numprocs, MPI_DOUBLE, MPI_SUM,
0, MPI_COMM_WORLD);
if (0 == myid) printf("\nInner product = %10.2f",res);
MPI_Finalize();
}
```

Распределение и сбор данных

При программировании часто возникает задача распределения массива данных по процессам некоторыми регулярными "кусками". Например, распределение матрицы, нарезанной вертикальными лентами. Возникает и обратная задача - сбор на некотором выделенном процессе некоторого набора данных, распределенного по всем процессам.



MPI_Scatter

Распределение и сбор данных осуществляется с помощью вызовов процедур MPI_Scatter и MPI_Gather.

int MPI_Scatter(void* sendbuf, int sentcount, MPI_Datatype senddatatype, void* recbuf, int reccount, MPI_Datatype recdatatype,int root,MPI_Comm comm)

– sendbuf - адрес буфера для передачи данных;
– sentcount - количество элементов, передаваемых на каждый процесс (общее количество элементов в буфере равно произведению sentcount на количество процессов в коммуникаторе);
– senddatatype - тип передаваемых данных;
– recbuf - буфер для приема данных;
– reccount - размер буфера recbuf;
– recdatatype - тип данных для приемки;
– root - ранг процесса, с которого рассылаются данные;
– comm – коммуникатор.

Как работает?

Процесс с рангом root произведет передачу данных всем другим процессам в коммуникаторе. Каждому процессу будет отправлено sendcount элементов. Процесс с рангом 0 получит порцию из sendbuf, начиная с 0-го и заканчивая sendcount-1 элементом. Процесс с рангом 1 получит порцию, начиная с sendcount, заканчивая 2* sendcount-1 и т.д.

Пример

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int send_buf[size];
    int recv_buf;

    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            send_buf[i] = i * i;
        }
    }

    MPI_Scatter(send_buf, 1, MPI_INT, &recv_buf, 1, MPI_INT, 0, MPI_COMM_WORLD);

    printf("Process %d received data: %d\n", rank, recv_buf);

    MPI_Finalize();

    return 0;
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вы
Process 1 received data: 1
Process 4 received data: 16
Process 5 received data: 25
Process 6 received data: 36
Process 7 received data: 49
Process 0 received data: 0
Process 2 received data: 4
Process 3 received data: 9
```

MPI_Gather

Подпрограмма MPI_Gather собирает данные от остальных процессов.

int MPI_Gather(void* sendbuf, int sentcount, MPI_Datatype senddatatype, void* recbuf, int reccount, MPI_Datatype recdatatype,int root,MPI_Comm comm)

– sendbuf - адрес буфера для передачи данных;
– sentcount - количество элементов, передаваемое на главный процесс;
– senddatatype - тип передаваемых данных;
– recbuf - буфер для приема данных;
– reccount - размер буфера recbuf;
– recdatatype - тип данных для приемки;
– root - ранг процесса, на котором собираются данные;
– comm – коммуникатор.

Как работает?

Посредством MPI_Gather каждый процесс в коммуникаторе передает данные из буфера sendbuf на процесс с рангом root. Этот "ведущий" процесс осуществляет склейку поступающих данных в буфере recbuf. Склейка данных осуществляется линейно, положение пришедшего фрагмента данных определяется рангом процесса, его приславшего. В целом процедура MPI_Gather обратна по своему действию процедуре MPI_Scatter.

Следует заметить, что при использовании MPI_Gather сборка осуществляется только на одном процессе. Во всех остальных процессах заполнение буфера recbuf не определено. Для некоторых задач необходимо, чтобы данные, рассчитанные на каждом из процессов, были собраны в единый объект опять же на каждом процессе. В таком случае, вместо функции MPI_Gather следует использовать функцию MPI_Allgather. При использовании функции MPI_Allgather на всех процессах в буфере recbuf будут собраны одинаковые данные - "большой" объект, полученный как объединение фрагментов, переданных с каждого из процессов.

Пример

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int send_buf = rank;
    int recv_buf[size];

    MPI_Gather(&send_buf, 1, MPI_INT, recv_buf, 1,
MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Process 0 received data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", recv_buf[i]);
        }
        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

```
vasiliy@BEAST:/mnt/d/Dropbox/lol/Вычислители
Process 0 received data: 0 1 2 3 4 5 6 7
```

В данном примере каждый процесс отправляет свое значение переменной send_buf в процесс 0, а затем процесс 0 собирает все эти значения в массив recv_buf и выводит их на экран.

Что?

Кроме перечисленных, в MPI существует еще десяток функций, осуществляющих различные коллективные операции. При работе с ними следует помнить следующие основные моменты:

– Все коллективные операции выполняются в рамках коммуникатора. Если необходимо выполнить коллективную операцию над подмножеством процессов, следует создать для этой цели свой коммуникатор.

– Коллективные операции должны вызываться во всех процессах, которые в них участвуют.

– Разумное использование коллективных операций - хорошее средство повышения производительности.

Набор коллективных операций.

- Синхронизацию всех процессов с помощью барьеров (MPI_Barrier).
- Коллективные коммуникационные операции, в число которых входят:
  - рассылка информации от одного процесса всем остальным членам некоторой области связи (MPI_Bcast);
  - сборка (gather) распределенного по процессам массива в один массив с сохранением его в адресном пространстве выделенного (root) процесса (MPI_Gather, MPI_Gatherv);
  - сборка (gather) распределенного массива в один массив с рассылкой его всем процессам некоторой области связи (MPI_Allgather, MPI_Allgatherv);
  - разбиение массива и рассылка его фрагментов (scatter) всем процессам области связи (MPI_Scatter, MPI_Scatterv);
  - совмещенная операция Scatter/Gather (All-to-All), каждый процесс делит данные из своего буфера передачи и разбрасывает фрагменты всем остальным процессам, одновременно собирая фрагменты, посланные другими процессами в свой буфер приема (MPI_Alltoall, MPI_Alltoallv).
- Глобальные вычислительные операции (sum, min, max и др.) над данными, расположенными в адресных пространствах различных процессов:
  - с сохранением результата в адресном пространстве одного процесса (MPI_Reduce);
  - с рассылкой результата всем процессам (MPI_Allreduce);
  - совмещенная операция Reduce/Scatter (MPI_Reduce_scatter);
  - префиксная редукция (MPI_Scan).

Пример использования MPI_Barrier

```c
#include <stdio.h>
#include <mpi.h>

#define ARRAY_SIZE 100

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int array[ARRAY_SIZE];
    int sum = 0;

    // Заполнение массива на каждом процессе
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = rank * ARRAY_SIZE + i;
    }

    // Расчет суммы элементов массива на каждом процессе
    for (int i = 0; i < ARRAY_SIZE; i++) {
        sum += array[i];
    }

    // Все процессы достигли этой точки и ждут друг друга
    MPI_Barrier(MPI_COMM_WORLD);

    // Процесс 0 собирает сумму элементов массива со всех процессов
    int total_sum = 0;
    MPI_Reduce(&sum, &total_sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Total sum: %d\n", total_sum);
    }

    MPI_Finalize();
    return 0;
}
```

Набор коллективных операций

- Коллективные коммуникации не взаимодействуют с коммуникациями типа точка-точка.
- Коллективные коммуникации выполняются в режиме с блокировкой. Возврат из подпрограммы в каждом процессе происходит тогда, когда его участие в коллективной операции завершилось, однако это не означает, что другие процессы завершили операцию.
- Количество получаемых данных должно быть равно количеству посланных данных.
- Типы элементов посылаемых и получаемых сообщений должны совпадать.
- Сообщения не имеют идентификаторов.

Спасибо за внимание!

